BAIL security

EULER
Auxilliary

FINAL REPORT

March '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not bue considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Euler – Auxilliary - Audit Report
Website	euler.finance
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/euler-xyz/euler-price-oracle/tree/6055a513adb688919ff40c09c9ec46b35df73499 ChainlinkInfrequentOracleXStocks.sol https://github.com/euler-xyz/evk-periphery/tree/d2b3925ea9373b72310f95d9a350f0715f5b7e20 HookTargetTermsOfUse.sol https://github.com/euler-xyz/evk-periphery/tree/eae25d382bb58aaaaa9f1a3a55567af993ce32be - SwapVerifier.sol
Resolution 1	https://github.com/euler-xyz/euler-price-oracle/tree/5c59a75c5e2a043ec0418e6b56833d54205fb6de ChainlinkInfrequentOracleXStocks.sol https://github.com/euler-xyz/evk-periphery/tree/28682195c0823a4ec1508caa2fb7e96b00816402 HookTargetTermsOfUse.sol
Resolution 2	https://github.com/euler-xyz/evk-periphery/blob/87b425bfc84dca50e30f26f8d5014c722e61c8d1/src/HookTarget/HookTargetTermsOfUse.sol

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High						
Medium	2	2				
Low						
Informational	9			9		
Governance	2			2		
Total	13	2		11		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

ChainlinkInfrequentOracleXStocks

The ChainlinkInfrequentOracleXStocks contract is an extension which is added towards the ChainlinkInfrequentOracle contract and it simply incorporates a safety check before the quoting mechanism. It is meant to be used for the following implementation contract:

https://github.com/backed-fi/backed-token-contract/blob/324555775de4683e16eef7265d3f8c311e92b5ec/contracts/BackedAutoFeeTokenImplementation.sol

Please note that the source code of this contract still may change. It would require a re-audit in this scenario.

Appendix: Multiplier Updates

xStocks tracks balances in internal shares, and uses a global 1e18-precision multiplier to convert shares into ERC-20 balances (balance = shares * multiplier / 1e18). The multiplier can be explicitly updated (optionally scheduled with an activationTime) by a privileged updater, and it can also drift over time due to periodic fee accrual computed in getCurrentMultiplier() and folded into storage via the updateMultiplier modifier.

Transfers, mints, and burns convert the user-facing token amount into shares using the current multiplier, then move shares, so any multiplier change rebases all holders proportionally.

The oracle essentially prevents disrupting updates with a safety mechanism.

Appendix: Pausing Periods

When quoting, the oracle iterates multiplier updates from newest to oldest. For updates in the future (A > t), it checks them only if they are within the “before” bracket (t + B >= A), and continues scanning. When it reaches the first past/current update (A <= t), it checks it only if it is within the “after” bracket (A + C >= t) and then breaks, skipping all older updates even if they are still inside the after window.

This gives a distinct period in which condition checks are executed and simply skips them if the update is too far in the future or if the period after the last “past update” has exceeded a certain threshold.

Privileged Functions

- none

Core Invariants:

INV 1: Multiplier checks must always start from the last multiplier

INV 2: Multiplier checks must only happen within the before and after periods

INV 3: Multiplier activationTime must be only ever increasing per index

INV 4: There can only ever be one index with future activationTime, based on XStocks implementation

Issue_01	Break-early bypass in _checkMultiplierUpdates() for multiple past updates
Severity	**Medium**
Description	_checkMultiplierUpdates() iterates multiplier updates from newest to oldest, but breaks immediately after processing the first past/current update. If there are multiple past updates whose activationTime values are still within pauseTimeAfter, the function will only evaluate the newest one and may skip an older significant update that should have triggered PriceOracle_MultiplierUpdatePause(): *if (activationTime + pauseTimeAfter >= block.timestamp) {* *_checkMultiplierChange(previousMultiplier, newMultiplier);* *}* *break;* The unconditional break assumes that once the "newest past update" is examined, all older updates are irrelevant. That assumption is only valid if you can prove: - there can be at most one past update within the entire pauseTimeAfter horizon, and/or - updates are spaced strictly more than pauseTimeAfter apart. Neither is enforced by the xStocks token implementation. Multiple updates can occur close in time (including the same block), and _storeScheduledMultiplierUpdate can record multiple past entries. This becomes especially problematic if the update which breaks the loop passes the condition check while the one which was before this "in time", would not pass the check. Assume: - pauseTimeAfter = 24 hours

	- maxAllowedMultiplierChange = 0.05e18 (5%)
	
	Token history contains two past updates within the last 24h:
	
	- Significant update (should pause):
	  - U1: previous = 1.00e18, new = 0.90e18 (10% change), activationTime = now - 23h
	- Small update (noise):
	  - U2: previous = 0.90e18, new = 0.899e18 (~0.11% change), activationTime = now - 1h
	
	Now query the oracle at t = now:
	
	- The loop starts from newest (U2).
	- U2.activationTime <= now ⇒ past/current branch.
	- U2.activationTime + pauseTimeAfter >= now ⇒ it checks U2 and does not revert (change < 5%).
	- Then it executes break; and stops.
	- U1 (10% change, still within after-window) is never checked ⇒ oracle does not revert even though it should.
	
	It has to be noted that even when this mechanism is changed, stepwise increases between changes combined with multiple changes (e.g. each change has 10%) within a specific time-period, would not be detected.

Recommendations	Consider whether it makes sense to refactor the condition mechanism incorporate more than just the most recent multiplier update which has activationTime <= block.timestamp
Comments / Resolution	Resolved.

Issue_02	Typo in contract file naming
Severity	**Informational**
Description	Currently, the contract file is named as ChainlinkInfrequentOraclXStocks. This is incorrect as it should be ChainlinkInfrequentOracleXStocks.
Recommendations	Consider renaming the file.
Comments / Resolution	Acknowledged.

Issue_03	Immutable variables might require changes
Severity	**Informational**
Description	Currently, the following variables are immutable: */// @notice Time bracket to pause before the multiplier update in seconds.* *uint256 immutable public pauseTimeBefore;* */// @notice Time bracket to pause after the multiplier update in seconds.* *uint256 immutable public pauseTimeAfter;* */// @notice Max relative multiplier change allowed without pausing (WAD).* *uint256 immutable public maxAllowedMultiplierChange;* Depending on business dynamics, it might actually be required to change these variables.
Recommendations	Consider whether it makes sense to make these variables mutable.
Comments / Resolution	Acknowledged.

Issue_04	Division by zero in case of previousMultiplier being zero
Severity	**Informational**
Description	The math of the deviation check is as follows: *diff * 1e18 / previousMultiplier* This will simply revert if previousMultiplier is set to zero.
Recommendations	Consider whether it could ever happen that the previousMultiplier is zero. In that scenario, the current arithmetic operation would not work, firstly, due to division by zero and secondly because an increase from zero to even 1 wei is already non-computable. We recommend thinking whether it makes sense to remain the logic blocked or implement a bypass of the check in that situation. This is a business logic decision which can however be likely acknowledged due to the fact that previousMultiplier will never be zero (unless there is a weird initialization setup)
Comments / Resolution	Acknowledged.

Issue_05	DoS in case of excessive loops
Severity	**Informational**
Description	Currently, there is no possibility to reach excessive loops, given the implementation of the multiplier settings: *if(multiplierUpdates[multiplierUpdates.length - 1].activationTime > block.timestamp) {* *multiplierUpdates.pop();* *}* However, it is theoretically possible via the initialize_v4 function: *function initialize_v4(* *MultiplierUpdate [] calldata _pastMultipliersUpdates* *) external virtual {* *require(multiplierUpdates.length == 0, "BackedAutoFeeTokenImplementation v4 already initialized");* *multiplierUpdates.push(MultiplierUpdate({* *previousMultiplier: 1e18,* *newMultiplier: 1e18,* *activationTime: 0* *}));* *for (uint i = 0; i < _pastMultipliersUpdates.length; i++) {* *multiplierUpdates.push(MultiplierUpdate({* *previousMultiplier: _pastMultipliersUpdates[i].previousMultiplier,* *newMultiplier: _pastMultipliersUpdates[i].newMultiplier,* *activationTime: _pastMultipliersUpdates[i].activationTime* *}));* *}* *}* As one can see, this function allows for adding multiple future multipliers which then could result in more than 2 loops during the check, reverting due to OOG in extreme cases.

	Notably, this specific function also allows for violating the strict-time-increasing sequence of multiplier updates (each next index should never have an activationTimestamp which is older than the previous multiplier update).
Recommendations	Consider acknowledging this issue and making sure that the initialization has already been finalized.
Comments / Resolution	Acknowledged.

Issue_06	Multiplier checks are not symmetric in terms of multiplicative deviation
Severity	**Informational**
Description	The implementation of the multiplier math check is as follows: diff * 1e18 / previousMultiplier This math is not properly reflecting absolute changes: Example A: previous=100, new=200: relative = 100% Example B: previous = 200, new = 100: relative = 50% As we can see, a decrease of 100 reflects only a 50% change while an increase of 100 reflects a 100% change. At the same time, this means that a setting of maxAllowedMultiplierChange >= 1e18 will limit multiplier increases but never catches multiplier decreases: Example: - previous = 1e18 - new = 0.000001e18 - diff = 999999000000000000 - 999999000000000000 * 1e18 / 1e18 = 999999000000000000 No catch is ever triggered. This is likely the intended design, however, out of caution we are making this note.
Recommendations	Consider acknowledging this behavior.
Comments / Resolution	Acknowledged.

HookTargetsTermsOfUse

The HookTargetsTermsOfUse contract is a hook contract which is invoked in two specific scenarios:

Scenario 1: Called by EVC

Scenario 2: Called by the Vault

It then executes a caller based identity check which extracts the relevant address. After the address has been extracted, the account owner of the address is fetched via the EVC - if there is no account owner, the account will be the address directly. Finally, it ensures that the address has signed the termsOfUseHash, via routing to the TermsOfUseSigner contract.

The contract owner can freely change the termsOfUseHash and can add and remove addresses to the bypassList set.

Appendix: Limitation of this scope

This review only covers the HookTargetsTermsOfUse contract. Due to the multiple different execution possibilities of Vault/EVC, it may be possible that the current architecture allows for bypassing TOS settings in specific control-flows or even worse: block different business logic executions if an address has deliberately not signed TOS. However, that is not subject to this audit, as it only covers the correctness of the Hook itself.

Appendix: Called by EVC

Whenever the EVC is the caller of the hook, the identity is fetched from the executionContext within the EVC contract.

Appendix: Called by Vault

Whenever the Vault is the caller of the hook, the identity check is fetched by extracting the last 20 bytes of the calldata.

Appendix: ExecutionContext

The EVC's execution context is the central coordination mechanism for the entire Euler vault ecosystem. It acts as an implicit parameter channel, rather than passing authentication data through function arguments down the call chain, the EVC writes the authenticated identity

and state flags into a single storage slot before calling external contracts, and those contracts read it back by calling into the EVC.

This design enables a decoupled architecture where vaults do not need to receive authentication data as explicit parameters, and the EVC does not need to know anything about vault-specific function signatures.

In the context of this audit this means that either the Vault will call back into the EVC to fetch the executionContext or the Hook will call back into the EVC to fetch the executionContext.

Privileged Functions

- setTermsOfUseHash
- addBypass
- removeBypass

Core Invariants:

INV 1: _checkTermsOfUse must return early if the address is whitelisted for bypassing the TOS check

INV 2: _checkTermsOfUse must fetch the account owner of the address for the TOS check

INV 3: If the address has no account owner, it must be directly used for the TOS check

Issue_07	Incorrect TOS check within checkVaultStatus
Severity	**Governance**
Description	The checkVaultStatus function invokes the callHookWithLock function: *callHookWithLock(vaultCache.hookedOps, OP_VAULT_STATUS_CHECK, address(evc));* As we can see, the last parameter is the EVC address, which is then used further: *function callHookWithLock(Flags hookedOps, uint32 operation, address caller) internal virtual {* *if (hookedOps.isNotSet(operation)) return;* *invokeHookTargetWithLock(caller);* *}* And it now explicitly invokes the hookTarget address directly (without any EVC context beforehand): *function invokeHookTarget(address caller) private {* *address hookTarget = vaultStorage.hookTarget;* *if (hookTarget == address(0)) revert E_OperationDisabled();* *(bool success, bytes memory data) = hookTarget.call(abi.encodePacked(msg.data, caller));* *if (!success) RevertBytes.revertBytes(data);* *}* Thus, the checkVaultStatus function will simply always revert.
Recommendations	Consider simply not using the OP_VAULT_STATUS_CHECK in conjunction with the hook execution.
Comments / Resolution	Acknowledged.

Issue_08	Governance Privilege: DoS of executions
Severity	**Governance**
Description	Currently, governance of this contract has several privileges for invoking certain functions that can drastically alter the contracts behavior. The contract owner can freely change the TOS which then results in reverts until the new TOS are signed.
Recommendations	Consider incorporating a Gnosis Multisignature contract as owner and ensuring that the Gnosis participants are trusted entities.
Comments / Resolution	Acknowledged.

Issue_09	Violator can deliberately DoS liquidation in case of TOS change
Severity	**Medium**
Description	EVK vaults can be configured to call an external hook target on specific operations (hookedOps). When an operation is hooked, the vault invokes the hook target via a low-level call using abi.encodePacked(msg.data, caller), where caller is the vault's authenticated active account. HookTargetTermsOfUse enforces Terms of Use acceptance in its fallback() by deriving the effective caller either from trailing calldata (when called by a factory-recognized vault proxy) or from EVC execution context (when called directly by EVC). In the vault→hook path, the hook extracts the last 20 bytes of calldata as the effective caller and checks whether that caller's EVC owner has signed the current termsOfUseHash. Separately, liquidations and collateral seizures are executed through EVC's controlCollateral, which interacts with a specific targetCollateral contract. Within the liquidation control-flow during seizure execution, corresponding to EVC.controlCollateral(collateralVault, borrowerAccount, ...), EVC sets the execution context's onBehalfOfAccount to borrowerAccount and calls the collateral vault with msg.sender = EVC. The collateral vault authenticates the active account as borrowerAccount and, if the called operation is hooked, passes borrowerAccount to the hook via trailing calldata. In HookTargetTermsOfUse, the effective caller for vault-triggered hooks is extracted from the last 20 bytes of calldata, so the hook checks TOS acceptance for the **borrower's EVC owner** rather than for the liquidator initiating controlCollateral. This allows for deliberate DoS of liquidations if governance just updated termsOfUseHash, previously compliant borrowers (signed old hash), which are now violators become non-compliant for the new hash until they sign again. As a result, liquidation/controlCollateral flows (specifically collateral

	seizure) that touch hooked operations can revert due to TermsOfUseNotSigned() for the borrower owner. The problem lies specifically within the transfer call in the executionContext of controlCollateral, where it now points to the violator during the executionContext: *if (haveCommonOwnerInternal(onBehalfOfAccount, msgSender) || targetContract == msg.sender || targetContract == address(this) || contextCache.isControlCollateralInProgress()) { executionContext = contextCache.setOnBehalfOfAccount(onBehalfOfAccount).clearOperatorAuthenticated(); } else { executionContext = contextCache.setOnBehalfOfAccount(onBehalfOfAccount).setOperatorAuthenticated();* *}* while at the same time, transfers are also used in other executionContexts, which means a transfer enforces TOS checks under all circumstances, usually with the correct executionContext. The bypass mechanism is not a practical mitigation for this case because the bypass list is checked against the borrower's owner identity; governance would need to bypass affected borrowers (potentially at scale) to restore liquidation functionality after a hash change. In fact, it requires that the liquidation and seizure control-flow carefully excludes TOS checks in that specific scenario, making sure that liquidation corresponding seizures are not hooked and allow for reverts under edge-cases. While this does not directly apply to the auditing scope, we intuitively searched for edge-cases due to TOS implementation and therefore stumbled over this issue.
Recommendations	Consider ensuring that the controlCollateral flow with the transfer

	execution does not execute a hook check. While this intuitively could be solved by implementing a check within the TOS requirement that bypasses in case of isControlCollateralInProgress, we are unable to fully verify this without having audited the full EVK mechanism, as further edge-cases may apply.
Comments / Resolution	Resolved, such a check has been implemented, however, it appears that this exposes a vulnerability which allows for bypassing the TOS check via all control-flows that are executed via controlCollateral, which includes depositing etc, simply by doing this workaround with a controller. Therefore, we refer to the simple recommendation of ensuring that such a transfer does not execute a hook call, which means the code change should be reversed.
Resolution 2	The previous strict enforcement has now been removed and it will be ensured that the hook setup is not triggered during liquidations.

Issue_10	Potentially unexpected outcomes due to caller identification pattern
Severity	**Informational**
Description	The caller identification pattern is based on the caller of the fallback function, these two scenarios are existing: A: Whenever the EVC is the caller of the hook, the identity is fetched from the executionContext within the EVC contract. B: Whenever the Vault is the caller of the hook, the identity check is fetched by extracting the last 20 bytes of the calldata. Since the EVC as well as the Vault contract incorporate their very own logic (executionContext setting and fetching; appending of 20 bytes to the calldata), it can happen that there are unexpected control-flows which may allow spoofing the context address. While via Vault calls, it is guaranteed that the appended caller address is the return value of getCurrentOnBehalfOfAccount, there might be edge-cases in the EVC which could allow for spoofing this return value. The exact same comment applies to the direct EVC -> Hook call, as it likewise consults the getCurrentOnBehalfOfAccount function. The executionContext therefore is the fundamental mechanism which must ensure correctness.
Recommendations	Consider elaborating such potential scenarios.
Comments / Resolution	Acknowledged.

Issue_11	Unexpected outcome due to onlyOwner modifier
Severity	**Informational**
Description	The onlyOwner modifier exists for setTermsOfUseHash, addBypass and removeBypass and the implementation is as follows: *function _checkOwner() internal view virtual {* *if (owner() != _msgSender()) {* *revert OwnableUnauthorizedAccount(_msgSender());* *}* *}* As one can see it consults the _msgSender function which then in turn uses the already described caller identity mechanism. While this seems to work properly for a direct call by the owner, there could also be the intention that these functions are called via the EVC (e.g. in batching) which then would follow the executionContext logic. While this is not in the auditing scope, it could be possible that there is some edge-case where the executionContext could be spoofed to then execute governance executions without actually being a privileged address. It also introduces a risk in case the eVaultFactory is untrusted as it now allows anyone to simply append 20 bytes to the calldata and spoof the onlyOwner check.
Recommendations	Consider incorporating this scenario and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_12	Unconventional edge-case within EVC.controlCollateral may allow for executing privileged actions via controller of governance
Severity	**Informational**
Description	In EVC.controlCollateral, EVC forwards calls to targetCollateral on behalf of onBehalfOfAccount, relying solely on the controller's authority (onlyController(onBehalfOfAccount)) and requiring that targetCollateral is present in the account's collateral set. During controlCollateral, EVC sets executionContext.onBehalfOfAccount = onBehalfOfAccount and explicitly clears operatorAuthenticated. If HookTargetTermsOfUse is called via this path, msg.sender at the hook target equals EVC and the hook target's _msgSender() resolves to executionContext.onBehalfOfAccount via EVCUtil._msgSender() and getCurrentOnBehalfOfAccount(). As a result, if onBehalfOfAccount is set to the hook target's Ownable owner address, the OpenZeppelin onlyOwner check passes even though the caller is the controller and not the owner. If the hook target contract is mistakenly enabled as a collateral for the hook target owner's EVC account, then the controller of that account can invoke onlyOwner functions through EVC.controlCollateral. Note that this requires the governance address to set an enabled controller and the hook target address would need to be incorrectly set as account collateral. This is almost impossible to reach in practice but in our opinion a note about this potential behavior is helpful. This also appears to be true for the normal operator mechanism but this seems to be fully intended and no explicit edge-case.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

SwapVerifier

The SwapVerifier contract is a simple implementation which is used in the EVK swap periphery. The scope of this audit is explicitly the verifyAmountMinAndDeposit function, which checks if the contract holds a specific minimum amount and then executes a simple transfer out.

It is expected that it is only called in a trusted environment where funds have been received in the SwapVerifier contract and are transferred out atomically to the expected recipient.

Privileged Functions

- none

Core Invariants:

INV 1: The contract must only be used in a trusted environment

INV 2: The contract should never store any funds for longer than the duration of a transaction

Issue_13	Idle funds can be withdrawn
Severity	**Informational**
Description	Currently, it is expected that the SwapVerifier contract is only invoked in a trusted environment where funds are transferred in and out atomically. If there are ever any idle funds, these can be withdrawn from the contract.
Recommendations	Consider only using this contract in a trusted environment.
Comments / Resolution	Acknowledged